MeTTaIL + MeTTaCore in Lean and Rust

A Focused Companion to `leanOSLF.tex`

Mettapedia Project Notes (Draft)

August 31, 2026

Abstract

MeTTaIL is a formally verified rewrite-rule language framework embedded in Lean 4. It provides a syntax for pattern-matching rewrite rules with typed premises, a declarative one-step semantics, a proved-sound-and-complete executable engine, and an automatic derivation of an OSLF behavioral type system for any `LanguageDef` instance. This note covers the MeTTa-facing slice of the formalization: the MeTTaIL and MeTTaCore Lean libraries, the MeTTaMinimal and MeTTaFull instance endpoints, the Rust execution bridge, and a machine-checked formalization of MQ-calculus (Stay & Meredith 2026) as a concrete verification target exercising the full stack. A TPTP/TSTP case study additionally shows how official syntax, binder-resolving FOF-to-CNF transformations, pluggable calculus services, finite word compilation, and an authored operational GSLT compose without turning a standard proof format into a hardcoded prover.

1 Current Coverage

Component	Lines	Sorries
`Mettapedia/OSLF/MeTTaIL/`	5,250	0
`Mettapedia/OSLF/MeTTaCore/`	4,666	0
`Mettapedia/Languages/MeTTa/HE/`	2,774	0
`Mettapedia/OSLF/PeTTa/`	3,921	0
`Mettapedia/Logic/GovernanceReasoning/` (MeTTa)	1,140	0
`Framework/MeTTaMinimalInstance.lean`	700	0
`Framework/MeTTaFullInstance.lean`	346	0
Total (MeTTa-focused Lean slice)	18,797	0

2 Lean Side: What Is Formalized

2.1 MeTTaIL Syntax and Rule Language

The syntax layer is the shared grammar from which all MeTTaIL language instances are built: any `LanguageDef` is assembled from these types, so getting them right is a prerequisite for correctness of every higher layer. Core syntax and rule structures (locally nameless patterns, premises, equations, rewrites, language definitions) are in `Mettapedia/OSLF/MeTTaIL/Syntax.lean`.

Listing 1: Core MeTTaIL structures (excerpt)

```
inductive CarrierKind where
```

```
  | ast | tokenLabel | tokenRaw | tokenProof
  | tokenPath | builtinInt | builtinString | builtinBool

structure TypeDecl where
  name : String
  carrier : CarrierKind := .ast

inductive Pattern where
  | bvar : Nat -> Pattern
  | fvar : String -> Pattern
  | apply : String -> List Pattern -> Pattern
  | lambda : Option String -> Pattern -> Pattern -- authored binder name
  | multiLambda : Nat -> List String -> Pattern -> Pattern
  | subst : Pattern -> Pattern -> Pattern
  | collection : CollType -> List Pattern -> Option String -> Pattern

inductive SyntaxPatternOp where -- Rust-style metasyntax
  | var : String -> SyntaxPatternOp
  | sep : String -> String -> Option SyntaxPatternOp -> SyntaxPatternOp
  | zip : String -> String -> SyntaxPatternOp
  | map : SyntaxPatternOp -> List String -> List SyntaxItem -> SyntaxPatternOp
  | opt : List SyntaxItem -> SyntaxPatternOp

inductive Premise where
  | freshness : FreshnessCondition -> Premise
  | congruence : Pattern -> Pattern -> Premise
  | relationQuery : String -> List Pattern -> Premise
  | forAll : String -> String -> Premise -> Premise -- quantified premises

structure LanguageDef where
  name : String
  types : List TypeDecl -- carrier-aware type declarations
  terms : List GrammarRule
  equations : List Equation
  rewrites : List RewriteRule
  logic : List LogicDecl := []
  oracles : List OracleDecl := []
  reflectivePresentations : List ReflectivePresentationDecl := []
```

Key design choices:

- **Carrier-aware types** (`TypeDecl` with `CarrierKind`): Metamath concrete syntax requires raw lexical carriers (`![raw] as Sym`, `![label] as Label`). Rust already supports these via `LangType.carrier_kind`; Lean now matches.
- **Authored binder names** (`Pattern.lambda : Option String`): The human-facing name is preserved for export and diagnostics. The locally nameless core uses `none`; proofs are unaffected.
- **Syntax operators** (`SyntaxPatternOp`): Rust's `*sep`, `*zip`, `*map`, `*opt` are first-class authored data, not flattened to terminal/nonTerminal strings.
- **Authored contextual closure** (`Premise.congruence`): a rewrite rule explicitly states each context in which reduction may recur. Collection representation alone grants no reduction

authority.

- **Quantified premises** (`Premise.forAll`): Required for scope-extrusion rules like `xs.*map(|x| x # ...rest)`.
- **Default congruence is empty**: Each language opts in explicitly; no hidden reduction behavior.
- **Semantic validation** (`LanguageDef.validate`): Checks for unknown types, duplicate constructors, unbound syntax parameters, unknown relations, and arity mismatches—all at definition time.

2.2 The `languageDef!` Authored Notation

The Rust side uses a `language!` procedural macro to let humans author language definitions in a single readable block. Lean now has a matching `languageDef!` macro that parses the *same* concrete syntax into the *same* `LanguageDef` type—no separate "syntax tree" is introduced. The macro *is* the notation layer; the single `LanguageDef` is the core.

Listing 2: Lean `languageDef!` example (`LanguageDefDSLTests.lean`)

```
def smokeLang : LanguageDef :=
  languageDef! {
    name : "SmokeLang"
    types {
      Expr
      Env
      Result
      ![raw] as Sym -- carrier-aware type
    }
    terms {
      SymLeaf . tok:Sym |- tok : Expr;
      Lam . ^ x . body : [Expr -> Expr] -- binder name preserved
        |- "lam" body : Expr;
    }
    equations {
      EqSym . tok:Sym |- SymLeaf(tok) = SymLeaf(tok);
    }
    rewrites {
      Beta . env:Env | knownSymbol(SymLeaf(tok))
        |- Lam(SymLeaf(tok)) ~> SymLeaf(tok);
    }
    logic {
      relation knownSymbol(Expr);
      rule "knownSymbol(x) <-- true";
    }
    oracles {
      external evalExpr(Expr, Env) -> Result;
    }
  }
```

The Rust and Lean term-declaration notations are intentionally identical:

Rust `language!`

```
PPar . ps:HashBag(Proc)
  |- "{" ps.*sep("|") "}" :  Proc;
```

Lean `languageDef!`

```
PPar . ps:HashBag(Proc)
  |- "{" ps.*sep("|") "}" :  Proc;
```

Three additional layers support the authored notation:

1. **Validation.** `LanguageDef.validate` checks types, constructors, relations, syntax parameters, and duplicates at definition time (kernel-checked by `native_decide` in test examples).
2. **Fail-fast core bridge.** `CoreSyntaxBridge.specToCoreLanguage` returns `Except String` and rejects unsupported features (e.g. `forAll` premises, non-AST carriers, syntax metasyntax) rather than silently erasing them.
3. **Faithful export.** `Export.renderLanguageWithUserSyntax` preserves authored binder names, `...rest` collection remainders, and syntax operators—no invented `C_` prefixes or synthetic binder names.

The overall architecture is a single pipeline with two host languages:

Human-Authored Source
Rust `language!` / Lean `languageDef!`

↓

ONE `LanguageDef`
(types, terms, equations, rewrites, logic, oracles)

↙ ↘

Rust Runtime	**Lean Verification**
MORK/MM2, Ascent, tree-sitter parser	Proofs, OSLF typing, validated export

2.3 RhoCalc as the `LanguageDef` Parity Pressure Test

The ρ-calculus (Meredith & Stay) is the flagship language instance for both the Rust `language!` macro and the Lean `languageDef!` notation. The Rust definition in `hyperon/mettail-rust/languages/src/rhocalc.rs` (∼380 lines) covers the full process calculus including native arithmetic, string operations, and fold terms.

Lean now has a directly authored process fragment (`Languages/ProcessCalculi/RhoCalculus/LanguageDefDSL.lean`) covering the pure process kernel:

Listing 3: RhoCalc process fragment in Lean `languageDef!` form (excerpt)

```
def rhoCalcProcessFragment : LanguageDef :=
  languageDef! {
    name : "RhoCalc"
    types { Proc; Name }
    terms {
      PZero . |- "{}" : Proc;
      PDrop . n:Name |- "*" "(" n ")" : Proc;
      PPar . ps:HashBag(Proc) |- "{" ps.*sep("|") "}" : Proc;
      POutput. n:Name, q:Proc |- n "!" "(" q ")" : Proc;
      PInputs. ns:Vec(Name), ^[xs].p:[Name* -> Proc]
        |- "(" *zip(ns,xs).*map(|n,x| n "?" x).*sep(",") ")"
```

```
        "." "{" p "}" : Proc;
    NQuote . p:Proc |- "@" "(" p ")" : Name;
    PNew . ^[xs].p:[Name* -> Proc]
      |- "new" "(" xs.*sep(",") ")" "in" "{" p "}" : Proc;
  }
  equations {
    QuoteDrop . N:Name |- NQuote(PDrop(N)) = N;
  }
  rewrites {
    Exec . P:Proc |- PDrop(NQuote(P)) ~> P;
    ParCong . | S ~> T
      |- PPar({S, ...rest}) ~> PPar({T, ...rest});
    NewCong . | S ~> T
      |- PNew(^[xs].S) ~> PNew(^[xs].T);
  }
}
```

All structural properties are kernel-checked: `LanguageDef.validate` returns `[]` (proved by `native_decide`), and export substring checks confirm that the Lean-rendered text matches the Rust syntax.

Honest boundary. Two Rust rules cannot yet be authored in Lean:

- **Comm** requires `*zip(ns,qs).*map(...)` in *pattern* position (not just syntax-pattern position) and a direct `(eval cont ...)` substitution form.
- **Extrude** requires `xs.*map(|x| x # ...rest)` in *premise* position—quantified freshness over a collection.

These are the next DSL extensions needed for full parity.

2.4 Authored Contextual Semantics and Compiler Equivalence

The biconditional between the declarative relation and the executable engine is the central correctness guarantee of MeTTaIL: it means the engine can be used for computation while the declarative relation is used for reasoning, and the two are interchangeable by theorem. The least one-step relation generated by authored rewrite rules is `ContextualStep.Step` in `Mettapedia/OSLF/MeTTaIL/ContextualStep.lean`. A `Premise.congruence` recursively invokes that same relation; collection representation carries no independent reduction authority. Its executable compiler is indexed by an explicit contextual-depth bound and is proved exact.

Listing 4: Least authored contextual semantics and exact compiler

```
inductive StepAt (base : BasePremiseEvaluator) (lang : LanguageDef) :
    Nat -> Pattern -> Pattern -> Prop where
  | rule : rule in lang.rewrites ->
      initial in matchPatternForRule lang rule source ->
      PremisesAt base lang fuel initial rule.premises final ->
      applyBindingsForRule lang rule final = target ->
      StepAt base lang (fuel + 1) source target

def Step (base : BasePremiseEvaluator) (lang : LanguageDef)
    (source target : Pattern) : Prop :=
  exists fuel, StepAt base lang fuel source target
```

```
theorem mem_rewriteAt_iff_stepAt :
    target in rewriteAt base lang fuel source <->
    StepAt base lang fuel source target

theorem exists_mem_rewriteAt_iff_step :
    (exists fuel, target in rewriteAt base lang fuel source) <->
    Step base lang source target
```

2.5 OSLF Synthesis Hooks Used by MeTTa Instances

Every `LanguageDef` automatically acquires a behavioral type system via `langOSLF`: the modal Galois connection $\Diamond \dashv \Box$ and the OSLF type system are derived, not hand-coded, which means language designers get behavioral typing for free. Language-level reduction, engine equivalence, Galois connection, and synthesized OSLF type system are in `Mettapedia/OSLF/Framework/TypeSynthesis.lean`.

Listing 5: TypeSynthesis bridge theorems

```
theorem langReducesUsing_iff_execUsing (relEnv : RelationEnv) (lang : LanguageDef)
    (p q : Pattern) :
    langReducesUsing relEnv lang p q <-> langReducesExecUsing relEnv lang p q

theorem langGaloisUsing (relEnv : RelationEnv) (lang : LanguageDef) :
    GaloisConnection (langDiamondUsing relEnv lang) (langBoxUsing relEnv lang)

def langOSLF (lang : LanguageDef) (procSort : String := "Proc") :
    OSLFTypeSystem (langRewriteSystem lang procSort)
```

2.6 MeTTaCore State Machine and Properties

MeTTaCore models the MeTTa runtime as an explicit state machine over multisets of atoms, giving precise machine-checked statements of progress and determinism for the grounded-operation layer. State and transition components are in `Mettapedia/OSLF/MeTTaCore/State.lean` and `Mettapedia/OSLF/MeTTaCore/RewriteRules.lean`; key properties are in `Mettapedia/OSLF/MeTTaCore/Properties.lean`.

Listing 6: Representative MeTTaCore properties

```
structure MeTTaState where
  input : Multiset Atom
  knowledge : Atomspace
  workspace : Multiset Atom
  output : Multiset Atom

theorem progress (s : MeTTaState) (a : Atom) (_ : a in s.workspace) :
    s.knowledge.insensitive a = true \/
    (s.knowledge.queryEquations a).card > 0

theorem grounded_confluence (op : String) (args : List Atom) :
    forall r1 r2,
    executeGroundedOp (.symbol op) args = some r1 ->
    executeGroundedOp (.symbol op) args = some r2 ->
```

```
    r1 = r2
```

2.7 HE MeTTa: Computable Recursive Interpreter

The Hyperon Experimental (HE) MeTTa semantics are formalized as a set of six mutually recursive fuel-bounded functions in `Mettapedia/Languages/MeTTa/HE/`, directly corresponding to `metta`, `interpret_expression`, `interpret_atom`, `interpret_symbol`, `interpret_type`, and `chain_results` from the HE specification.

Listing 7: HE MeTTa interpreter core (excerpt from Interpreter.lean)

```
mutual
  def mettaCall (fuel : Nat) (space : Space) (bindings : Bindings)
      (atom : Atom) (typ : Atom) : ResultSet
  def interpretExpression (fuel : Nat) (space : Space) (bindings : Bindings)
      (atom : Atom) (typ : Atom) : ResultSet
  def interpretAtom (fuel : Nat) (space : Space) (bindings : Bindings)
      (atom : Atom) (typ : Atom) : ResultSet
  -- ... (6 mutually recursive functions)
end
```

Supporting modules formalize the full HE runtime:

- `Types.lean`: ErrorCode, Bindings, ResultSet
- `Space.lean`: Space, queryEquations, GroundedDispatch
- `Matching.lean`: matchAtoms, mergeBindings
- `TypeCheck.lean`: typeCast

Conformance. 37 kernel-checked conformance theorems in `Conformance.lean` verify concrete evaluations against expected results, all closed by `rfl` (no axioms, no `native_decide`).

OSLF pipeline export. The HE interpreter is also exported as an OSLF `LanguageDef` (`HELanguageDef.lean`) and a `PremiseProgram` (`HEPremises.lean`), enabling automatic generation of the Rust Ascent backend. The `PremiseProgram` specifies 19 premise relations and their datalog rules as first-order data that the Lean exporter renders to Ascent syntax.

2.8 PeTTa: Logic-Programming MeTTa Semantics

PeTTa formalizes MeTTa evaluation through a Prolog-inspired logic-programming lens. The core is `MeTTaEval`, an inductive big-step evaluation relation in `Mettapedia/OSLF/PeTTa/MeTTaEval.lean`:

Listing 8: MeTTaEval inductive relation (excerpt)

```
inductive MeTTaEval : PeTTaSpace -> Pattern -> Atom -> List Pattern -> Prop where
  | symbolLookup (sp : PeTTaSpace) (sym : String) (results : List Pattern) :
      sp.queryEquations (.symbol sym) = results ->
      results != [] ->
      MeTTaEval sp (.symbol sym) undefinedType results
  | groundedDispatch ...
  | expressionInterpretation ...
```

This relation is the shared core that both HE and PeTTa agree on at the answer level: the refinement proofs (Section 2.10) show that PeTTa refines HE evaluation.

Additional PeTTa modules include:

- `LPSoundness.lean`: LP-engine soundness for rewrite-only evaluation
- `GroundedOracle.lean`: typed grounded-operation interface
- `TypedEval.lean` / `TypeSystem.lean`: type-enriched evaluation
- `Effects.lean` / `MinimalInstructions.lean`: effect tracking and instruction set
- `TranslateExpr.lean`: expression-to-pattern translation

2.9 Metaprogramming and Dialect Translation

An experimental HE↔PeTTa translator (`hyperon/translators/`) revealed a fundamental difference in how the three MeTTa runtimes handle code-as-data during structural rewriting. All three runtimes can load source atoms into a space and use `match` to inspect them without evaluation. However, when a match template *reconstructs* the matched term with rewritten subexpressions, the three runtimes diverge:

Runtime	Output of chain→let rewrite	Variable handling
CeTTa (C)	`(= (sq $x) (* (+ $x 1) (+ $x 1)))`	Unified/inlined
HE (Rust)	`(= (sq $x) (* (+ $x 1) (+ $x 1)))`	Unified/inlined
PeTTa (Prolog)	`(= (sq $x) (let $y (+ $x 1) (* $y $y)))`	Preserved

All three outputs are **semantically equivalent** (they compute the same function), but PeTTa produces the **syntactically faithful** translation while HE/CeTTa produce a **beta-reduced** expansion. The cause: MeTTa's `match` performs full unification across the entire pattern. When a template variable `$v` binds to a source variable `$y`, all occurrences of `$y` in the template are also unified, effectively inlining the binding. PeTTa's Prolog-based matching uses `copy_term` and fresh variable generation, which preserves syntactic structure.

Implications.

- Pattern mining, backward chaining, and other metaprogramming tasks that require structural inspection of code are naturally easier in PeTTa. This may explain why PeTTa's pattern miner finds all results where HE's has edge-case failures.
- For exact source-to-source translation, the translator should be written in PeTTa (primary) with a semantic-equivalence version in HE/CeTTa.
- A future MeTTa-Pure could address this with an explicit quoting mechanism, de Bruijn indices, or meta-level separation (as formalized in the ρ-calculus layer of MeTTaIL).

Bidirectional Prolog translator. The practical translator is implemented in SWI-Prolog (`hyperon/translators/he\_to\_petta.pl`, `petta\_to\_he.pl`), where term manipulation is native and variable scoping is handled correctly by Prolog's `copy_term`. Each translation rule is a single, auditable `translate_term/2` clause:

Direction	Rule	Clause
HE→PeTTa	`chain`→`let`	`translate_term([chain,E,V,B], [let,V,TE,TB])`
HE→PeTTa	`collapse-bind`→`collapse`	`translate_term([collapse-bind,X], [collapse,TX])`
HE→PeTTa	`superpose-bind`→`superpose`	`translate_term([superpose-bind,X], [superpose,TX])`
HE→PeTTa	`switch`→`case`	`translate_term([switch,S,B], [case,TS,B])`
HE→PeTTa	`atom-subst`→`let`	`translate_term([atom-subst,A,V,T], [let,V,TA,TT])`
HE→PeTTa	`function/return`	unwrap
HE→PeTTa	`nop`→`let $_`	side-effect wrapper
PeTTa→HE	`progn`→nested `let $_`	2- and 3-argument forms
PeTTa→HE	`prog1`→`let+let`	save-first pattern
PeTTa→HE	`@<`→`<s`	string comparison

All rules recursively translate subexpressions via `maplist(translate_term)`; shared constructs (`if`, `let`, `case`, `match`, `superpose`, `collapse`, arithmetic) pass through unchanged.

The translator is validated on the full corpus of real `.metta` files:

Direction	Files tested	Passed
HE→PeTTa (CeTTa test suite)	93	93
PeTTa→HE (PeTTa examples + miner)	344	344
Total	**437**	**437**

A correct-by-construction MeTTa S-expression parser (`hyperon/translators/metta\_parser.pl`) handles all MeTTa token types: `$variables`, `%types%`, `"strings"`, numbers, operators, and nested S-expressions. The lexer rule is minimal: tokens are delimited by whitespace and parentheses only.

Towards a pure MeTTa translator. The Prolog translator works but lives outside MeTTa. A pure MeTTa implementation would close the bootstrapping loop (MeTTa translating MeTTa). The challenge documented in `metaprogramming_challenges.metta` is that MeTTa's unification-based `match` prevents exact syntactic rewriting. PeTTa's Prolog backend naturally avoids this via `copy_term`, so a PeTTa-native translator using `import_prolog_functions_from_file` is the most promising path. This would let PeTTa call the Prolog translator rules directly from MeTTa code, combining MeTTa's space-based program loading with Prolog's faithful term manipulation.

The translator corpus is at `hyperon/translators/`, with `metaprogramming_challenges.metta` documenting five approaches and their failure modes.

2.10 Governance Refinement Proofs

The governance reasoning layer (`Mettapedia/Logic/GovernanceReasoning/`) connects MeTTa evaluation to deontic temporal logic via DDL+. This is the end-to-end chain: evaluate MeTTa code → decode results → interpret as deontic modalities → apply DTS theorems.

Shared interface. `LetStarInterface.lean` defines a `MeTTaLike` typeclass parametric over any evaluator supporting rewrite-rule application. Both HE and PeTTa instantiate it, so let* unfolding theorems are proved once and work for both implementations.

HE refinement. `HERefinement.lean` bridges `MeTTaEval` to the governance DTS obligations with explicit type/binding tracking. Seven semantic refinement theorems are proved, including let* integration (`he_letStar_full_2`, `he_letStar_full_3`).

PeTTa refinement. `PeTTaRefinement.lean` establishes the PeTTa-side DTS refinement and let* unfolding at the PeTTa level.

HE canonical conformance. `HECanonicalConformance.lean` connects the canonical computable HE interpreter to governance DTS theorems. Seven conformance theorems prove end-to-end: HE eval produces correct OB⇒PE results for concrete governance scenarios (soaMoor, payRent, giveNotice), all closed by `rfl` or `simp`.

All files: 0 sorries, 0 custom axioms.

2.11 MeTTaMinimal and MeTTaFull Instance Endpoints

These are the integration points where abstract MeTTaIL theorems become concrete MeTTa-language guarantees: a developer who adds a new MeTTa feature can check its behavioral soundness by extending one of these instances. MeTTa instances export checker-soundness theorems with instance-specific atom interpretations:

Listing 9: MeTTa instance API endpoints

```
def mettaMinimalOSLF := langOSLF mettaMinimal "State"
theorem mettaMinimal_checkLangUsing_sat_sound_specAtoms :
    checkLangUsing ... = .sat -> sem ...

def mettaFull : LanguageDef := ...
def mettaFullOSLF := langOSLF mettaFull "State"
def mettaFullRelEnv : RelationEnv := ...
theorem mettaFull_checkLangUsing_sat_sound_specAtoms :
    checkLangUsing ... = .sat -> sem ...
```

3 Type System Limitation: No Subject Reduction

3.1 Subject Reduction in Type Theory

A type system satisfies *subject reduction* (also called *type preservation*) if typing is preserved under evaluation: whenever $a : T$ and T reduces to T', we have $a : T'$. Subject reduction is a foundational theorem in well-behaved dependent type theories—it holds for Martin-Löf Type Theory, the Calculus of Constructions, and Lean 4 itself.

3.2 MeTTa's Typing Judgment

MeTTa's type system, formalized in `Mettapedia/OSLF/MeTTaCore/Types.lean`, uses a *purely lookup-based* judgment:

Listing 10: HasType judgment (excerpt from Types.lean)

```
inductive HasType (space : Atomspace) : Atom -> Atom -> Prop where
  | intrinsicSymbol (s : String) :
      HasType space (.symbol s) (.symbol "Symbol")
  | annotated (a ty : Atom) :
      typeAnnotation a ty in space.atoms ->
      HasType space a ty
  ...

def typeAnnotation (a ty : Atom) : Atom :=
  .expression [.symbol ":", a, ty]
```

The critical rule is `annotated`: the only way a term a acquires a non-intrinsic type T is for the literal annotation `(: a T)` to be present in the atomspace. There is no inference rule that propagates types through evaluation of the type expression itself.

3.3 The Theorem

`Mettapedia/OSLF/MeTTaCore/SubjectReduction.lean` establishes:

Listing 11: Subject reduction definitions and main theorem

```
def SubjectReduction
    (HasTy : Atomspace -> Atom -> Atom -> Prop)
    (Reduces : Atomspace -> Atom -> Atom -> Prop) : Prop :=
  forall (space : Atomspace) (a T T' : Atom),
    HasTy space a T -> Reduces space T T' -> HasTy space a T'

-- One-step syntactic rewrite: (= a b) in space.atoms
def AtomReduces (space : Atomspace) (a b : Atom) : Prop :=
  .expression [.symbol "=", a, b] in space.atoms

theorem metta_not_subject_reduction :
    not (SubjectReduction HasType AtomReduces)
```

The proof is by explicit counterexample. Define the atomspace

$$\mathcal{S} = \{\ \texttt{(: t (Vec n))},\ \texttt{(= (Vec n) (Vec 0))}\ \}.$$

Three lemmas, each closed by kernel-checked computation (`decide`):

1. t : `(Vec n)` in $\mathcal{S}$, via `HasType.annotated`.
2. `(Vec n)` reduces to `(Vec 0)` in $\mathcal{S}$, since `(= (Vec n) (Vec 0))` $\in \mathcal{S}$.
3. $t \not:$ `(Vec 0)` in $\mathcal{S}$: the only surviving case of the `HasType` induction is `annotated`, which would require `(: t (Vec 0))` $\in \mathcal{S}$—but the space contains no such atom. Lean 4 closes all other constructor cases automatically via index discrimination.

The file also establishes the companion result `counterSpace_not_eval_closed`: the counterexample space fails the *eval-closure* property—the atomspace-level condition that *would* suffice for subject reduction, namely that whenever `(: a T)` $\in \mathcal{S}$ and T reduces to T', also `(: a T')` $\in \mathcal{S}$.

3.4 Significance

This result is not a bug in MeTTa; it is a design consequence of separating *knowledge* (rewrite rules and annotations, stored in the atomspace) from *computation* (evaluation via pattern matching). In MeTTa, types and terms are both atoms evaluated by the same engine. The engine can reduce a type expression, but the typing relation does not track this: `HasType` reflects the atomspace at a fixed moment, not under its closure.

This contrasts with MLTT/CoC, where subject reduction is proved by induction on the typing derivation: every rule carries a proof term that is transformed, not just looked up, when the type reduces. MeTTa's expressiveness (Turing-complete rewriting over a uniform atom language) comes at the cost of this metatheoretic guarantee. Eval-closure of the atomspace is a sufficient—though not necessary—runtime condition for subject reduction to hold in a given MeTTa program.

4 Rust Bridge Status (`hyperon/mettail-rust`)

The Rust side implements three MeTTa language backends, all registered in the REPL and augmented with the meta-introspection and Python oracle systems.

4.1 Language Backends

- **MeTTaHE** (780 lines): `languages/src/mettahe\_from\_lean.rs`—the HE MeTTa backend, generated from the Lean OSLF pipeline (`HELanguageDef.lean` + `HEPremises.lean`). Defines 44 term constructors across 4 types (State, Instr, Atom, Space), 17 rewrite rules, and 19 premise relations implemented as Ascent datalog rules. Eight builtin helper functions provide structural dispatchers for grounded arithmetic, equation lookup, type annotation search, and function-type resolution.
- **MeTTaFullState** (1,618 lines, legacy): `languages/src/mettafull\_legacy.rs`—the original MeTTaFull backend with hand-written premise rules and `eval_ground_call_core` recursive evaluator. Retained for regression testing; the HE backend supersedes it for HE-conformant evaluation.
- **PeTTa** (226 lines): `languages/src/petta\_from\_lean.rs`—currently delegates to MeTTaFullState with library aliases. Planned for standalone reimplementation from Lean PeTTa specs.

All three are registered in `repl/src/registry.rs` with the `OracleAugmentedLanguage` wrapper, which adds the meta-introspection oracle (types/terms/rewrites/relations counts and membership queries) and the gated Python oracle.

4.2 Premise Program IR

The MeTTaHE backend demonstrates the `PremiseProgram` pipeline: premise semantics are specified as first-order datalog rules in Lean, then rendered to Ascent syntax for the Rust backend. Negation over derived relations (forbidden by Ascent stratification) is handled via positive complement relations (e.g., `typeNotMatchesMetaOrAtom` instead of `!typeMatchesMetaOrAtom`).

Macro hygiene note. The `language!` proc macro generates parser functions with internal variables (`pos`, `lhs`, `rhs`, `value`, etc.). Term constructor field names that collide with these produce confusing type errors. The HE backend documents the required renames in its file header.

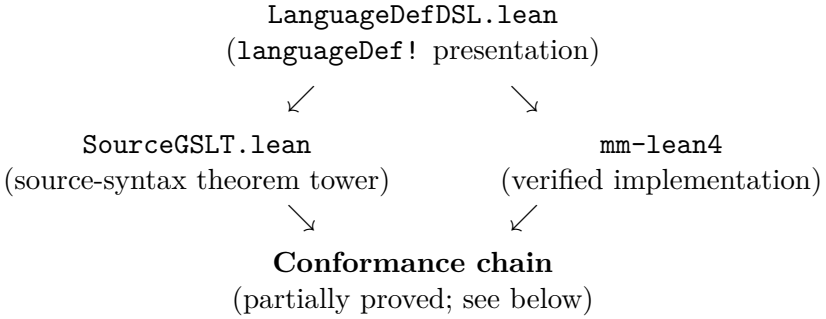
4.3 Current Passing Rust Tests

Listing 12: MeTTa Rust canaries (all passing)

```
relation_query_can_bind_fresh_output_vars
typecheck_uses_space_typing_relation
grounded_call_boolean_not_reduces
return_step_reaches_done_state
default_registry_contains_mettafull
```

4.4 Metamath as a Three-Way Conformance Target

Metamath is the first language where three independently developed artifacts are connected through `LanguageDef`:



The `languageDef!` presentation `metamathCore` defines the full Metamath statement syntax—`Database`, `Stmt`, `MathString`, `ProofString`, normal and compressed proofs, blocks, and includes—with carrier-aware type declarations (`![label] as Label`, `![raw] as Sym`, `![proofTok] as ProofTok`, `![path] as IncludePath`) and explicit term constructors for every `$c`, `$v`, `$d`, `$f`, `$e`, `$a`, `$p` declaration form. Beyond syntax, its `rewrites` block authors the front-end lowering, environment-loading, linearization, and compilation pipeline as rewrite rules in the same language object, and the definition renders back to the `language!` concrete syntax with kernel-checked guards on the exported text. (A Rust `metamath.rs` twin existed historically; the rendered text is now checked directly, with no in-tree Rust consumer.)

A second, elaborated-record presentation `sourceGrammar` carries the proof-facing tower: it is validated by the generic well-formation checker (`sourceGrammar_valid`), declares its five lexical token classes with byte charsets, and every accepted run of the composed source pipeline derives the outer database sort in the lexicalized language of its own classified stream, with no externally supplied lexical or ordered-token premises (`runSource_selfLexicalized_derives`).

The `mm-lean4` project provides a complete, sorry-free, axiom-free verification of the Metamath proof checker:

- `DeclarativeSpec.lean`: `CN` (constants), `VR` (variables), `Expr` (expressions), `Provable` (proof validity).
- End-to-end biconditional: `verify_parser_acceptance_iff_spec_provable`—raw bytes $\leftrightarrow$ `Spec.Provable`, with 0 sorries and 0 axioms.

The types correspond across all three artifacts:

`languageDef!` form	`sourceGrammar`	mm-lean4 `DeclarativeSpec`
`Sym` (carrier: raw)	`symbol_tok` class	`CN` / `VR`
`Label` (carrier: label)	`label_tok` class	labels in frame
`MathString`	`symbol_list` rules	`Expr`
`Database`	`database` rules	parsed DB structure
`ProofString`	`proof_list` rules	proof witness

What is proved and what remains. The conformance chain between the source presentation and `mm-lean4` is now proved at the declaration level: every accepted source transition (`$c`, `$v`, `$d`, `$f`, `$e`, `$a`, `$p` insertion, scope push/pop/completion) is matched by the shipped database operation it corresponds to, and the whole statement fold co-evolves with the implementation database from the initial states with no supplied premises (`default_initial_applyPayloads`). This includes identifying the implementation's mandatory-frame computation with the specification's mandatory frame (`trimFrame'_eq_mandatory`). Three joints remain before the chain is a single bidirectional theorem: binding the composed pipeline's classified stream to the exact byte presentation the verified reader consumes; discharging each `$p` obligation's normal or compressed proof from the reader's own chronology, with verified and incomplete (`?`) outcomes kept distinct; and the prefix-wise simulation between the source fold and the production reader, in which reader microsteps are administrative and completed statements are the observable steps. These are in active development, with the normal-proof discharge recently constructed against the production reader trace. Separately, a statement-fragment translation between `metamathCore` and `sourceGrammar` would close the triangle's left edge; it is scoped but not yet started.

5 MQ-Calculus as a Formalized Verification Target

MQ-calculus (Stay & Meredith 2026) is a process calculus in which communication events are measurement events: a synchronization on wire i is not just a handshake but a quantum measurement, selecting a continuation branch according to Born-rule probabilities. The grammar is deliberately minimal—six constructors covering nil, parallel composition, fresh-wire allocation, gate application, output, and measurement-branching input—which makes the full formalization tractable while still exercising the semantically interesting cases.

MQ-calculus is included in this formalization as a concrete stress test for the MeTTaIL stack. It is not a special case; it is an instance of `LanguageDef` and inherits the full MeTTaIL pipeline automatically: the syntax is given by the `Process` inductive, the reduction relation `Reduces` is derived from `CommReduction` and structural congruence `SC`, the denotational semantics `denoteWith` is backend-parametric over `MQSemanticsBackend`, and the MORK cross-calculus coherence theorem (`paper_mork_mq_branching_equiv`) connects MQ COMM branching to MORK binary fold branching at the theorem level.

Three results distinguish the Lean formalization from a pencil-and-paper treatment. First, the Born-rule normalization theorem `born_prob_sum_one` is a machine-checked proof from Mathlib's `PiLp.norm_sq_eq_of_L2`, not an axiom. Second, the denotational semantics is parameterized by an abstract `MQSemanticsBackend`, so the same six denotational clauses are provably correct for any compliant backend without re-proving them. Third, every clause of §6.3 of the paper has a named Lean theorem in `PaperMap.lean` (see Appendix B), giving an explicit audit map that connects narrative paper text to kernel-checked declarations.

The MQ formalization also provides realistic workload profiles for `hyperon/mettail-rust`:

COMM-heavy processes exercise nondeterministic branching and collection-heavy parallel composition, which are the known Ascent scaling pressure points documented in the engine performance notes.

6 MeTTa-IL as a 2-Mode Adjoint Doctrine (MaTT)

The OSLF framework establishes that MeTTa-IL is an instance of a *2-mode adjoint doctrine fragment* in the sense of fibrational/doctrinal logic (Lawvere hyperdoctrines, Jacobs fibred category theory). This is the formal content of "MeTTa-IL is MaTT."

Theoretical position. MeTTa-IL-via-OSLF is not the syntactic multimodal type theory of [1] (which would require modal type formers $\Box_\mu A$, locks, and context extension). It is instead the *categorical/doctrinal* flavour: a mode category over a base of operational languages, equipped with a contravariant indexed predicate functor and a modal adjunction $\Diamond \dashv \Box$ at each behavioral mode. This places it squarely in the tradition of Lawvere hyperdoctrines and fibrational semantics for modal predicate logic.

What is proved. Nine claims are machine-checked. The capstone definition `mettaIL_2modeDocFrag : TwoModeAdjointDocFrag` (`MATTFragment.lean`) packages all of them into a single kernel-checked Lean object; the individual components are listed in Table 2.

Claim	Lean declaration	Sta
Per-language modal Galois connection	`doctrine_galois_is_langGalois`	pro
Per-language modal adjunction	`doctrine_adjunction_is_langModalAdjunction`	pro
Eq-category pullback functoriality	`eqCategory_mapPred_functorial`	pro
Mode-skeleton composition/predicate laws	`mode2SkeletonLaws`	pro
Runtime/runtime commuting square	`runtime_runtime_square_coherence`	pro
Runtime/behavioral commuting square	`runtime_behavioral_square_coherence`	pro
Runtime morphism diamond witness transport	`runtime_mode_diamond_transport_comp`	pro
Pure mode isolation	`pure_mode_isolation`	pro
mettaPure runtime→behavioral transport	`mettaPure_runtime_behavioral_transport`	pro
Capstone: MeTTa-IL is `TwoModeAdjointDocFrag`	`mettaIL_2modeDocFrag`	**pro**
Full mode-2-category (2-cells/2-morphisms)	—	out
Syntactic MTT ($\Box_\mu A$, locks, context ext.)	—	out
Pure-mode morphism theory	—	def

Table 2: MaTT claim map (doctrinal/fibrational sense). The proved-count is machine-checked: `provenCount_eq : countByStatus .proven = 9` (`MATTClaimMap.lean`, closed by `decide`).

The capstone theorem. A `TwoModeAdjointDocFrag` packages: (i) a mode category (identity, associativity laws for {`pure`, `runtime`(L), `behavioral`(L)}); (ii) a contravariant indexed predicate functor (`ModeHom.mapPred`, with identity and composition laws); (iii) a modal adjunction at each behavioral mode ($\Diamond \dashv \Box$, the OSLF Galois connection); and (iv) Beck-Chevalley commuting squares connecting the mode-level and category-level predicate pullbacks. The definition `mettaIL_2modeDocFrag : TwoModeAdjointDocFrag` fills all fields from existing proved lemmas.

The witness theorem `mettaIL_is_2mode_adjoint_doctrine : Nonempty TwoModeAdjointDocFrag` is the Lean-kernel-checked statement that this object exists.

Scope boundary. The out-of-scope items are explicit design choices. Full 2-category structure requires 2-cells (natural transformations between mode morphisms) and is deferred. Syntactic MTT modal type formers require a full dependent kernel with mode-indexed context extension; this is the role of MeTTa-Pure (Appendix A). Pure-mode morphism theory is deferred pending the MeTTa-Pure bridge.

Reproducibility.

Listing 13: Build command for MaTT targets

```
lake build \
  Mettapedia.OSLF.Framework.MATTFragment \
  Mettapedia.OSLF.Framework.MATTClaimMap \
  Mettapedia.OSLF.Framework.MATTProvableNow
```

The reviewer anchor in `Mettapedia/OSLF/SpecIndex.lean`:265 is labelled "Conservative MaTT Claim Map (No Overclaim)".

7 The CertificateGSLT Layer: Generic Proof Checking over Presentations

Beyond rewriting, a `LanguageDef` may declare *judgment forms* and *inference rules*. The generic inference checker (`Mettapedia/GSLT/LanguageDef/InferenceChecker.lean`) validates these proof-calculus fields — V1 fixes exact metavariable occurrence depths with no implicit lifting; V2 adds finite structural judgment signatures — and checks untrusted proof trees against them, reconstructing `Type`-valued `Derivation` evidence. Judgments are ordinary `Pattern` terms of the same syntax IR, so the proof lane introduces no second syntax authority.

The CertificateGSLT modules (`Mettapedia/GSLT/LanguageDef/`, canonical import `CertificateGSLTFramewo` compiled fixtures in `CertificateGSLTInterpretationCanary`) develop the categorical theory of these validated presentations:

- a strict rule-retaining preorder whose arrows transport derivations, checked proofs, and identical chronological proof-DAG artifacts, with a compiled negative theorem for missing rules;
- open derivations with positional premise holes and a proved simultaneous-substitution calculus (both unit laws, associativity);
- a provably non-thin category of derivation-valued interpretations — one primitive rule may be implemented by a composite target proof — with a compiled counterexample showing strict retention cannot express this;
- set-valued models with contravariant pullback and semantic-transport naturality, plus covariant indexed derivation families with concrete categories of elements;
- fuel-free checkers for open proof trees and chronological proof DAGs (premise references plus backward node references) with *exact* typed reconstruction, and a compiled sharing canary — two submitted DAG nodes, three expanded rule occurrences — keeping compact evidence identity distinct from expanded proof meaning;

- sharing-preserving substitution of checked DAGs into the premise holes of a checked DAG: acceptance preserved, expansion commutes with composition exactly, submitted node count exactly additive while expanded cost multiplies per citation, with compiled refuters for the unshifted and uniformly wired variants;
- the multisorted-clone structure of open derivations (judgments as sorts, proof plugging as simultaneous substitution), the algebraic core of a cartesian multicategory.

The standalone treatment, with the full theorem inventory and verification statement, is `CertificateGSLT.tex`. For this companion the operative point is that any validated presentation whose `LanguageDef` declares judgment forms and inference rules is already a CertificateGSLT object: the proof calculus, interpretation category, models, and both checked evidence formats come from the framework with no per-language authorship. The `languageDef!` macro does not yet author the proof-calculus fields; extending it is a natural seam alongside the next steps below.

8 TPTP and TSTP as Composed GSLTs

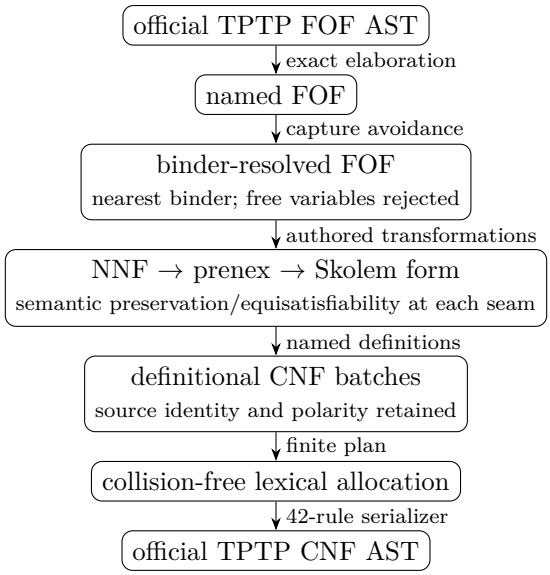
The TPTP language family is a useful stress test for the preceding claims. It combines a large versioned concrete grammar, binder-sensitive logical syntax, semantics-changing transformations such as Skolemization, and TSTP derivation DAGs whose inference annotations name many different calculi [2, 3]. The key architectural separation is:

> TPTP owns the standard syntax, formula roles, source annotations, and derivation graph. It does not own one privileged proof-search strategy or one universal inference calculus.

Accordingly, `lib_tptp` is a format, transformation, and verification library. Resolution search remains an ordinary MeTTa program in the examples tree. A TSTP inference is accepted only by an explicitly selected calculus service carrying its own semantic soundness theorem.

8.1 FOF to official CNF: transformations are languages

The current first-order pipeline is not one opaque clausifier. Every intermediate object is a typed carrier, and every computational arrow is an authored `LanguageDef` connected to an independent recursive semantics by exact agreement and no-invention theorems:



For example, the concrete source fixture

Listing 14: Nested shadowing at the official FOF boundary

```
fof(shadowing,axiom,![X]:(p(X)=>?[X]:q(X))).
```

is elaborated to named FOF and then to a locally nameless formula in which the occurrence in `q(X)` resolves to the inner existential binder, not the outer universal binder. The composed theorem `endToEndExact_exists` then obtains an exact official-CNF-AST serialization result for every evidence-bearing admitted batch. Its proof passes source-readiness obligations through binder resolution, NNF, de Bruijn shifting, prenexing, Skolemization, definitional naming, and lexical allocation. Numeric terms with arguments and nullary defined predicates are negative controls: they are proved outside the serialization domain rather than silently rendered.

This endpoint is deliberately an *official abstract syntax tree*. A canonical AST-to-concrete-text printer and parser/printer round trip are still separate obligations; the theorem does not call an AST "a `.p` file."

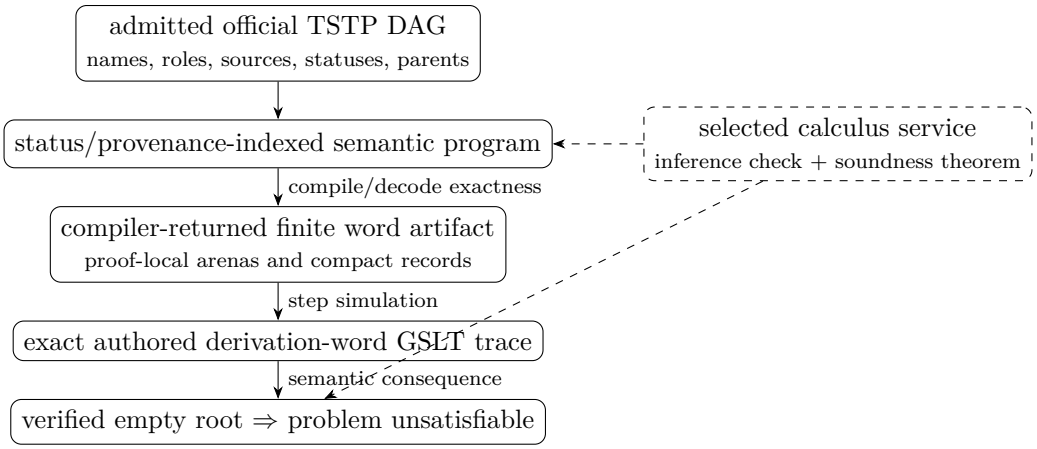
8.2 TSTP derivations: graph format outside, calculus inside

The first closed verification witness is ground resolution. The admitted fixture uses ordinary TSTP inference annotations:

Listing 15: A small status-carrying TSTP refutation

```
cnf(p_or_q,axiom,p|q).
cnf(not_p,axiom,~p).
cnf(not_q,axiom,~q).
cnf(q,plain,q,inference(resolution,[status(thm)],[p_or_q,not_p])).
cnf(empty,plain,$false,
    inference(resolution,[status(thm)],[q,not_q])).
```

The verification architecture keeps the TSTP status and calculus meanings distinct:



The generic finite compiler allocates arenas for formulas, rules, evidence, provenance, obligations, and service state, then emits seven compact records for this canary. `wordArtifact_decodes` recovers the exact semantic program. A program-local state-stability theorem supplies the dynamic arena obligation. The shared execution invariant proves a continuous exact rewrite trace in the actual authored derivation-word `LanguageDef`; `authoredTrace_unsatisfiable` derives unsatisfiability from the calculus service's independent soundness invariant. Out-of-arena formula references, malformed parent counts, and unknown opcodes are rejected.

The binary layer is thus a compact representation of a derivation checker, not another handwritten logical authority. It also exposes the intended native compilation shape: a compiler-returned artifact is retained and reused, rather than rebuilding finite arenas by normalization in every client theorem.

Current boundary. The closed semantic result covers this official ground-resolution profile. It does not yet verify arbitrary TSTP rule names, `status(cth)`, or `status(esa)`. General TSTP verification requires a registry of declared calculus services and a status-sensitive composition law; typed TFF/TCF/THF/NHF carriers, include-DAG resolution, external E/Vampire proof reconstruction, concrete printing, and generated StructuredC execution remain separate work. Whole-language validation also needs compositional per-row certificates so unchanged presentations are not repeatedly renormalized.

9 What This Companion Covers and Does Not Cover

Covered now:

- MeTTaIL syntax + declarative/executable semantics bridge (sound and complete).
- `languageDef!` macro with block-syntax parsing, carrier-aware types, authored binder preservation, syntax metasyntax operators, semantic validation, fail-fast core bridge, and faithful export.
- RhoCalc and Metamath as acceptance-test `LanguageDef` instances (process fragment and core-step fragment, respectively).
- MeTTaCore state machine + core progress/confluence-facing properties.
- MeTTaMinimal/MeTTaFull OSLF instance endpoints.

- HE MeTTa: computable recursive interpreter, 37 conformance theorems, OSLF `LanguageDef` + `PremiseProgram` export.
- PeTTa: inductive big-step evaluation, LP soundness, grounded oracle, typed evaluation, effect tracking.
- Governance refinement: shared `MeTTaLike` interface, HE/PeTTa DTS bridges, let* unfolding, canonical conformance (OB⇒PE).
- Executable Rust backends: MeTTaHE (from Lean pipeline), MeTTaFullState (legacy), PeTTa (alias wrapper), all with REPL + oracle integration.
- MeTTa-IL as 2-mode adjoint doctrine fragment (MaTT, 9 proved claims).
- Generic inference checker over `LanguageDef` judgments/rules, with the CertificateGSLT categorical layer (strict refinement, open derivations, non-thin interpretations, models, exact DAG evidence); Section 7 and the standalone `CertificateGSLT.tex`.
- TPTP FOF elaboration through binder resolution, NNF, prenexing, Skolemization, definitional CNF, collision-free allocation, and exact official-CNF-AST serialization; plus one status-indexed official ground TSTP refutation compiled to and executed by the authored derivation-word GSLT (Section 8).

Not yet full end-to-end by construction:

- Full RhoCalc `languageDef!` parity: COMM and Extrude require premise-side `*map(...)` and pattern-side `*zip(...).*map(...)`.
- Metamath `LanguageDef`-to-`mm-lean4` conformance theorem (type correspondence exists; semantic bridge is future work).
- Fully automatic Lean-to-Rust export without any manual adaptation (field-name hygiene and builtin rendering still require hand-editing).
- Standalone PeTTa Rust backend (currently delegates to MeTTaFullState).
- Full HE test suite validation against the Rust MeTTaHE backend.
- Full Hyperon atomspace/runtime integration (beyond encoded relation environment subset).
- General TSTP calculus-service coverage, typed TFF/TCF/THF/NHF semantics, concrete TPTP printing and round trips, E/Vampire proof reconstruction, and generated native execution of the derivation-word checker.

10 Immediate Next Steps

1. Extend `languageDef!` with premise-side `*map(...)` and pattern-side `*zip(...).*map(...)` to close the RhoCalc COMM/Extrude gap.
2. Prove the Metamath conformance bridge: `LanguageDef` rewrite semantics agree with `mm-lean4`'s `verify_parser_acceptance_iff_spec_provable`.
3. Run the HE test suite against the Rust MeTTaHE backend to validate semantic correctness of the generated premise rules.

4. Build a standalone PeTTa Rust backend from the Lean PeTTa specs, replacing the current MeTTaFullState delegation.

5. Fix the `language!` proc macro to use hygienic variable names in generated parser code, preventing field-name collisions.

6. Replace whole-language validation normalization in the derivation machines with compositional per-row certificates and transport.

7. Generalize the TSTP word compiler to finite evolving service-state arenas, then instantiate further status-sensitive calculus services.

8. Complete concrete TPTP printing and E/Vampire TSTP reconstruction before extending the semantic pipeline through TFF and THF.

9. Tie this note into `leanOSLF.tex` as a companion reference once stable.

A WIP: Toward MeTTa-Pure — A Dependent Kernel for MeTTa

This appendix sketches a forward-looking architecture for adding a trusted dependent type theory (DTT) layer to MeTTa without changing the semantics of existing MeTTa programs. It is work-in-progress design notes, not a finalized specification.

A.1 Motivation: Two Results in This Paper

Section 3 proves that MeTTa's current `HasType` judgment does not satisfy subject reduction. This is not a bug; it is a precise consequence of a design choice: `HasType` is annotation lookup, not inductive inference. No single-point patch can fix this while preserving the rest of MeTTa's open-world rewriting semantics.

The productive response is architectural, not remedial: *build a small dependent kernel alongside MeTTa, not inside it.*

A.2 The Four-Layer Stack

A clean MeTTa-DTT separates concerns across four layers:

Layer	Role
A — MeTTa-Pure	A tiny intensional DTT kernel: ordered contexts Γ, Π/Σ-types, identity types, universes, inductive families. Bidirectionally checked. Subject reduction and decidable conversion hold by construction.
B — Elaboration boundary	A (not necessarily trusted) elaborator translating MeTTa concrete syntax into kernel terms. A checked reflection bridge maps quoted MeTTa atoms to kernel terms and back.
C — Full MeTTa runtime	Current MeTTa unchanged: equations, atomspaces, `match`, `superpose`, `collapse`, grounded interop, spaces, nondeterminism, concurrency. The AGI language proper.
D — OSLF overlay	Behavioral/spatial/modal types derived from the operational semantics of Layer C, *not* from kernel conversion. The modal adjunction $\Diamond \dashv \Box$ and the presheaf/topos lift remain exactly here.

The core thesis: **DTT for the trusted structural core; OSLF for behavior; full MeTTa for open-world agent computation.**

A.3 Why MeTTaIL Is the Right Substrate

The kernel should be *specified and implemented in MeTTaIL*, not beside it. MeTTaIL already provides:

- locally nameless binder representation with cofinite quantification;
- proved substitution lemmas (`subst_open`, `subst_lc`, etc.);
- least authored contextual relation `ContextualStep.Step`, with an exact fuel-indexed compiler and no representation-wide descent rule;
- OSLF `langOSLF` synthesis from any `LanguageDef`.

A MeTTa-Pure `LanguageDef` is therefore a *language instance* of the existing MeTTaIL framework, not a new implementation. The `mettaPureOSLF` type system would be a fourth instance alongside `mettaMinimalOSLF`, `mettaFullOSLF`, and `tinyMLOSLF`.

A.4 Kernel Judgment Forms

The kernel uses separate judgment forms distinguished from atomspace typing:

Listing 16: Proposed MeTTa-Pure judgment forms

```
-- Well-formed type at universe level i
Sk ; G |- A type i

-- Synthesis: infer type of t
Sk ; G |- t => A
```

```
-- Checking: verify t against A
Sk ; G |- t <= A

-- Definitional equality
Sk ; G |- t == u : A
```

The local context `G` is an *ordered list* of typed variables `[(x1,A1), ..., (xn,An)]`—not an unordered atomspace. The signature `Sk` carries declared kernel constants.

A.5 Concrete Syntax (S-Expression Style)

The kernel interface can remain MeTTa-shaped:

Listing 17: Proposed MeTTa-Pure source forms

```
(Pi (($x A)) B) -- dependent function type
(lam (($x A)) t) -- lambda abstraction
(app t u) -- application
(Sg (($x A)) B) -- dependent pair type
(pair a b) -- pair introduction
(fst p) (snd p) -- pair elimination
(Id A a b) -- identity type
(refl a) -- reflexivity
(Type 0) (Type 1) ... -- universe hierarchy
```

This keeps the S-expression aesthetic while elaborating to a real binder AST. At Layer C, `=` stays the runtime rewrite operator; the syntax `:` annotation gains a second meaning as a kernel declaration when inside a pure context.

A.6 Kernel Conversion: What Is and Is Not Allowed

Kernel definitional equality is deliberately minimal:

- β: $(\lambda x.\, t)\, u \equiv t[u/x]$ for Π/Σ eliminations.
- δ: transparent kernel constant unfolding.
- ι: reduction rules for inductive eliminators.
- ζ: local `let`-unfolding (optional, guarded by termination).
- η: function and pair extensionality, only where clearly desired.

Not in kernel conversion:

- Arbitrary user-defined `=` rewrite rules (Layer C only).
- Atomspace lookup or mutation.
- `match`/`superpose`/`collapse` forms.
- Python/native grounded calls.

If rewrite rules are later admitted to definitional equality, this requires a separate certified rewrite subsystem with a machine-checked confluence/termination certificate—the $\lambda\Pi$-modulo route (Dedukti/Saillard)—as a distinct upgrade, not a default.

A.7 The "Don'ts"

Four constraints characterize the kernel boundary:

1. **Do not** let arbitrary MeTTa rewrite rules define kernel conversion.
2. **Do not** use an unordered atomspace as the kernel context.
3. **Do not** identify OSLF native types with dependent types; the two layers are orthogonal.
4. **Do not** allow full runtime nondeterministic evaluation inside types; type-level computation must terminate.

A.8 Categorical Summary

The three-part categorical picture is:

Layer	Categorical home
Kernel (MeTTa-Pure)	Categories with Families (CwF); contextual categories; locally cartesian closed categories; presheaf and groupoid models.
Full MeTTa / OSLF	Presheaf/topos/modal/fibrational semantics over the operational rewrite system; the $\Diamond \dashv \Box$ modal adjunction.
Bridge	Quotation and checked reflection between kernel terms and full MeTTa atoms.

The presheaf topos semantics of OSLF (already in the Mettapedia formalization) wraps a CwF kernel—exactly the structure that makes the categorical split coherent rather than arbitrary.

A.9 Implementation Reference Points

The closest existing architectures are:

- **Lean 4**: small total kernel, bidirectional elaborator, erasure for runtime performance; `partial` definitions are kernel-opaque.
- **Isabelle/Pure**: tiny LF-style framework underneath object logics (HOL, ZF, ...). Pure is itself a minimal DTT.
- **Isabelle/Spartan**: DTT as an object logic inside Pure, directly analogous to MeTTa-Pure as an object logic inside MeTTaIL.

In one sentence: *a clean MeTTa-DTT is the smallest conservative extension of MeTTaIL that adds a CwF-style dependent kernel with decidable conversion and subject reduction, while treating today's MeTTa as an effectful reflective metalanguage over that kernel, and OSLF as the behavioral logic of the resulting operational world.*

B MQ-Calculus Lean Walkthrough

This appendix maps paper-level MQ clauses to concrete Lean definitions. All listings are excerpted verbatim from the formalization.

Files.

- `MQCalculus/Syntax.lean` — process grammar and gate types
- `MQCalculus/CommRule.lean` — Born-rule probability and COMM rule
- `MQCalculus/Reduction.lean` — full structural reduction relation
- `MQCalculus/Backend.lean` — abstract semantic backend interface
- `MQCalculus/Denotational.lean` — backend-parametric denotation
- `MQCalculus/PaperMap.lean` — stable audit map (paper → theorem)

A. Process grammar and Born-rule reduction

Listing 18: Process grammar (`Syntax.lean`) and COMM rule (`CommRule.lean`)

```
-- Syntax.lean
inductive Process : Type where
  | MQNil : Process
  | MQPar : Process -> Process -> Process
  | MQNu : Process -> Process -- new-wire binder
  | MQGate : GateSpec -> Process -> Process -- typed gate, then P
  | MQOut : Nat -> Process -- emit wire i
  | MQIn : Nat -> Process -> Process -> Process -- measure i: 0 -> P, 1 -> Q

-- CommRule.lean
-- n-qubit Hilbert space as EuclideanSpace over Fin (2^n)
abbrev QVec (n : Nat) := EuclideanSpace Complex (Fin (2 ^ n))

-- Born-rule probability |<r|psi>|^2, proved (not axiomatized) from Mathlib
noncomputable def born_prob (r : Outcome) (psi : QVec 1) : Real :=
  match r with
  | .zero => Complex.normSq (inner ket0 psi)
  | .one => Complex.normSq (inner ket1 psi)

-- Probabilities sum to 1 for unit-norm states (proved from PiLp.norm_sq_eq_of_L2)
theorem born_prob_sum_one (psi : QVec 1) (h : norm psi = 1) :
    born_prob .zero psi + born_prob .one psi = 1

-- COMM rule: MQOut i | MQIn i P Q can branch to either P or Q
inductive CommReduction : Nat -> Process -> Process -> MeasurementBranch -> Prop where
  | outcome_zero (i : Nat) (p q : Process) :
      CommReduction i p q { outcome := .zero, result := p }
  | outcome_one (i : Nat) (p q : Process) :
      CommReduction i p q { outcome := .one, result := q }
```

B. Reduction relation and backend-parametric denotation

Listing 19: Reduction (`Reduction.lean`) and denotation (`Denotational.lean`)

```
-- Reduction.lean
-- Full structural reduction: COMM fires under SC, parallel, nu, gate contexts
```

```
inductive Reduces : Process -> Process -> Prop where
  | comm (i : Nat) (p q : Process) (b : MeasurementBranch) :
      CommReduction i p q b ->
      Reduces (MQPar (MQOut i) (MQIn i p q)) b.result
  | sc (p p' q' q : Process) :
      SC p p' -> Reduces p' q' -> SC q' q -> Reduces p q
  | par_l (p p' q : Process) : Reduces p p' -> Reduces (MQPar p q) (MQPar p' q)
  | par_r (p q q' : Process) : Reduces q q' -> Reduces (MQPar p q) (MQPar p q')
  | nu_step (p p' : Process) : Reduces p p' -> Reduces (MQNu p) (MQNu p')
  | gate_step (g : GateSpec) (p p' : Process) :
      Reduces p p' -> Reduces (MQGate g p) (MQGate g p')

-- MQOut i is irreducible (cannot step)
theorem mq_out_irreducible (i : Nat) {q : Process} (h : Reduces (MQOut i) q) : False

-- Multi-step with notation p ->* q
inductive MultiStep : Process -> Process -> Prop where
  | refl (p : Process) : MultiStep p p
  | step (p q r : Process) : Reduces p q -> MultiStep q r -> MultiStep p r

-- Denotational.lean
-- Backend-parametric denotation: same 6 clauses, any compliant backend
noncomputable def denoteWith (backend : MQSemanticsBackend) :
    Process -> QState -> Option QState
  | .MQNil, st => some st
  | .MQOut _, st => some st
  | .MQGate g p, st => denoteWith backend p (backend.applyGate g st)
  | .MQNu p, st => denoteWith backend p (backend.allocFresh st)
  | .MQPar p q, st => (denoteWith backend p st).bind (denoteWith backend q)
  | .MQIn i p q, st =>
      if backend.branchProb i .zero st >= backend.branchProb i .one st then
        denoteWith backend p (backend.collapse i .zero st)
      else
        denoteWith backend q (backend.collapse i .one st)
```

C. Paper-map audit map

`PaperMap.lean` exposes one named theorem per paper clause. To audit a specific claim, import the file and check the corresponding theorem.

Listing 20: Stable paper-to-theorem index (`PaperMap.lean`)

```
-- Section 6.3 denotational clauses (all rfl proofs from simp lemmas)
theorem paper63_nil : denoteWith backend MQNil st = some st
theorem paper63_out : denoteWith backend (MQOut i) st = some st
theorem paper63_gate : denoteWith backend (MQGate g p) st =
                         denoteWith backend p (backend.applyGate g st)
theorem paper63_new : denoteWith backend (MQNu p) st =
                         denoteWith backend p (backend.allocFresh st)
theorem paper63_par : denoteWith backend (MQPar p q) st =
                         (denoteWith backend p st).bind (denoteWith backend q)
theorem paper63_in : denoteWith backend (MQIn i p q) st =
                         if backend.branchProb i .zero st >= ... then ...
```

```
-- Norm-1 preservation under collapse (statevector backend, positive mass)
theorem paper63_statevector_collapse_norm_one_of_raw_pos
    (hraw : 0 < rawOutcomeProb i out st) :
    norm (statevectorBackend.collapse i out st).psi = 1

-- COMM denotation equals explicit branch-state selection
theorem paper63_comm_denote_eq_measurement_selection (i : Nat) (p q : Process) (st :
    QState) :
    denote (MQIn i p q) st = ...

-- Section 3 communication reduction facts
theorem paper3_comm_both_outcomes (i : Nat) (p q : Process) :
    CommReduction i p q {.zero, p} /\ CommReduction i p q {.one, q}

theorem paper3_comm_exhaustive (i : Nat) (p q : Process) (b : MeasurementBranch) :
    CommReduction i p q b -> b = {.zero, p} \/ b = {.one, q}

-- Cross-calculus coherence: MQ COMM branching <-> MORK binary fold branching
theorem paper_mork_mq_branching_equiv (i : Nat) (p q : Process) :
    (CommReduction i p q {.zero, p} /\ CommReduction i p q {.one, q}) <->
    (forall (fold : MORK.FoldStep) (_ : fold.isBinary), ...)
```

References

[1] D. Gratzer, G. A. Kavvos, A. Nordvall Forsberg, and L. Birkedal. Multimodal dependent type theory. In *Proceedings of the 35th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS 2020)*, pages 492–506. ACM, 2020.

[2] G. Sutcliffe. The TPTP language and syntax BNF. TPTP World user documentation, `https://tptp.org/UserDocs/TPTPLanguage/TPTPLanguage.shtml`.

[3] G. Sutcliffe. The TPTP format for derivations. TPTP World quick guide, `https://tptp.org/UserDocs/QuickGuide/Derivations.html`.